ATELIER 8 L'adresse introuvable

Algorithmie comparative - J4 - Jeudi 12 novembre 2026, 10h00 - 45 minutes, en binôme

La question à laquelle l'atelier répond

Le support tape "ste catherine 33000 bordeaux" et ne trouve pas la rue Sainte-Catherine (ticket T-0488). Comment rapprocher une adresse mal saisie du référentiel, vite et juste ?

Ce que vous produisez : adr/GRILLE-ADR-005.md et adr/ADR-005-adresses.md (carnet), quatre CSV dans bench/resultats : la force brute, les deux index, l'exactitude, la construction.

Le rendu attendu

En fin d'atelier : le code, les mesures et la grille passent le correcteur, et l'ADR existe en premier jet (chaque rubrique en une à trois lignes, avec vos chiffres et le nom de vos CSV). Le correcteur sans KO avant 17h00, le commit du binôme à 17h00. Ensuite, l'ADR s'améliore chez vous jusqu'au dépôt du carnet, le 1er décembre.

Avant 17h00, le correcteur de l'atelier répond sans KO :

```
OK  À COMPLÉTER : l'index de trigrammes retrouve les adresses
OK  Le coût d'une saisie est mesuré pour les trois variantes
OK  L'exactitude des trois variantes est mesurée
OK  Le coût de construction de l'index est mesuré jusqu'au référentiel entier
OK  La grille comparative de l'ADR-005 est remplie
OK  L'ADR-005 a ses six rubriques, remplies
OK  La rubrique 3 de l'ADR-005 renvoie à une mesure qui existe, la rubrique 5 donne le domaine de validité
OK  Les trois phrases de l'ADR-005 tiennent sans jargon
8 contrôles, 0 KO
```

Avant de commencer. maj-J4.zip décompressé et npm install lancé ce matin (sinon : annexe). Créez l'ADR-005 du binôme, que vous remplissez dès le sous-objectif 1 :

Windows	`Copy-Item adr\ADR-000-gabarit.md adr\ADR-005-adresses.md`
macOS et Linux	`cp adr/ADR-000-gabarit.md adr/ADR-005-adresses.md`

Pour chaque sous-objectif, la première commande et ce que vous devez voir ; les suivantes sont dans le README du dépôt (tableau des commandes de la journée 4) et dans les fiches atelier numéro 5 à 7, celles de mardi.

Les sous-objectifs

1. Reproduire le ticket T-0488 consigne 1 de la slide

Pourquoi. Voir l'erreur avant de chercher la vitesse : ce que répond chaque méthode à la saisie du support.

Première action. Lancez la recherche avec la saisie du ticket, puis relevez en rubrique 1 de l'ADR-005 ce que répond chaque méthode.

Les trois systèmes	`npm run exactitude -- --saisie "ste catherine 33000 bordeaux"`

Vous devez voir. La force brute du legacy rend la rue Héron, en quelques secondes ; PostgreSQL rend la rue Sainte-Catherine. La ligne trigrammes répond "rien" : son index est le À COMPLÉTER du sous-objectif 2. Relancez cette commande après le sous-objectif 2 : elle rend aussi la rue Sainte-Catherine.

```
  brute         1502 ms  -> 110 rue heron 33000 bordeaux
  postgres       343 ms  -> 2 rue sainte catherine 33000 bordeaux
```

2. Compléter l'index de trigrammes, lancer les tests consigne 2 de la slide

Pourquoi. L'index compte, pour chaque adresse, les trigrammes qu'elle partage avec la saisie : une addition au lieu d'une distance d'édition. C'est le seul À COMPLÉTER de l'atelier.

Première action. Complétez la méthode candidats de bench/variantes/adresses.ts : comptez les trigrammes communs, gardez les meilleures adresses, remettez chaque compteur à zéro. Puis lancez npm test.

Windows	`code bench\variantes\adresses.ts`
macOS et Linux	`code bench/variantes/adresses.ts`

Vous devez voir. Les trois tests des adresses passent. Deux tests échouent encore : celui des listes d'adjacence attend l'atelier 9, celui de Dijkstra avec tas l'atelier 10.

```
✔ les trigrammes suivent la règle de pg_trgm (0.8862ms)
✔ une saisie exacte retrouve son adresse, une saisie fautive une adresse proche (244.007ms)
✔ l'index en mémoire et PostgreSQL proposent les mêmes candidats (1230.7383ms)
```

3. Mesurer le coût d'une saisie consigne 3 de la slide

Pourquoi. Le critère du ticket est le temps d'une saisie. La force brute met des secondes par saisie : une à trois saisies suffisent.

Première action. Mesurez la force brute, puis les deux index sur 10 à 50 saisies (commande suivante du README). Divisez chaque durée par le nombre de saisies.

Les trois systèmes	`npm run bench -- --sujet adresses --variantes brute --tailles 1,2,3 --repetitions 3`

Vous devez voir. Des secondes par saisie pour la force brute. Vos durées seront différentes, l'écart avec les index non.

```
variante        taille   duree_ms   memoire_ko
brute                1    2087.83         5377
brute                2    5495.39         1670
brute                3    7737.85        63496
```

4. Mesurer l'exactitude et la construction consigne 4 de la slide

Pourquoi. Une méthode rapide qui se trompe livre au mauvais endroit : l'exactitude se mesure à part, sur trois cents saisies, par sorte de saisie.

Première action. Lancez la mesure d'exactitude (deux minutes), puis le coût de construction, sujet adresses-index. La taille de chaque index se lit dans la sortie de la mesure d'exactitude, à la fin de sa ligne (colonne index_mo du CSV), jamais dans memoire_ko.

Les trois systèmes	`npm run exactitude`

Vous devez voir. Les deux index retrouvent la même part des saisies et ratent les mêmes, les sans numéro : ces nombres sont les mêmes sur tous les postes. La taille de l'index est en fin de ligne, "index … Mo". La force brute n'est essayée que sur dix saisies : son score ne dit rien de sa robustesse. Notez-la sur le ticket T-0488, qu'elle rate.

```
  trigrammes  288/300 justes (96,0 %), 8,2 ms par requête, construction 1,4 s, index 50,8 Mo en mémoire
  postgres    288/300 justes (96,0 %), 142,0 ms par requête, construction 6,1 s, index 42,2 Mo sur le
disque de PostgreSQL
```

5. Remplir la grille, écrire l'ADR-005, vérifier consignes 5, 6 de la slide

Pourquoi. Les poids du ticket : temps 3, mémoire 1, lisibilité 1, robustesse 3, coût 2. Le temps se note pour une saisie sur la Gironde, trois fois plus d'adresses : 5 sous 0,1 s, 4 sous 1 s, 3 sous 5 s, 2 sous 10 s, 1 au-delà. La mémoire, celle ajoutée au processus du site : 5 si rien, 4 sous 500 Mo, 3 sous 2 Go, 2 sous 8 Go, 1 au-delà. La lisibilité, la robustesse et le coût ont chacun leur barème, en annexe.

Première action. Copiez la grille, notez les trois options, puis écrivez l'ADR-005 : sa rubrique 5 dit ce que vous n'avez pas pu mesurer, le PostgreSQL d'Efrei. Préparez la décision et la question de compréhension à deux, puis lancez `npm run verifier -- 8`.

Windows	`Copy-Item adr\grille-comparative.md adr\GRILLE-ADR-005.md`
macOS et Linux	`cp adr/grille-comparative.md adr/GRILLE-ADR-005.md`

- En atelier, l'ADR est un premier jet ; ce qui manque se reprend cet après-midi, dans le créneau de l'atelier 10, puis vous la finissez chez vous, pour le carnet du 1er décembre.
- Rubrique 3 : remplacez les colonnes du tableau du gabarit par une saisie (ms), saisies justes sur 300, taille de l'index, construction sur 229 329 adresses.
- La Gironde : vous avez mesuré sur la métropole ; multipliez par trois le temps d'une saisie et la taille de l'index de trigrammes. C'est une estimation haute pour les deux index : dites-le en rubrique 5. L'index de PostgreSQL est rangé sur le serveur de la base : il n'ajoute rien au processus du site, même si PGlite tourne dans Node en atelier.
- Rubrique 6 : ni unité (ms, Mo), ni nom de méthode ou de structure ; le correcteur donne d'un coup tous les mots qui coincent.

Vous devez voir. Le correcteur répond sans KO, comme en première page. Il vérifie l'index, les trois mesures, la grille et l'ADR ; il ne dit pas si la décision est bonne.

La décision de l'atelier

Quelle façon de rapprocher les adresses retenez-vous, et quel critère a tranché, avec un chiffre mesuré ? Les deux index sont défendables : la rubrique 5 de l'ADR dit pourquoi l'autre ne l'est pas, ou pas encore.

La question de compréhension

À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux : en fin d'atelier, l'intervenant passe la demander à quelques binômes.

À 17h00, le binôme commite : git add -A puis git commit -m "J4". Sans commit, le travail du jour n'est pas évalué.

Annexe, à consulter au besoin

Les cas particuliers

maj-J4.zip n'est pas décompressé, ou npm install n'a pas tourné (la commande s'arrête sur Cannot find package '@electric-sql/pglite') :

Windows	`Expand-Archive -Path $HOME\Downloads\maj-J4.zip -DestinationPath . -Force` `npm install`
macOS et Linux	`unzip -o ~/Downloads/maj-J4.zip -d .` `npm install`

Le référentiel d'adresses est absent (npm run exactitude le dit) : copiez l'archive adresses-33.csv.gz de Moodle (section Journée 2) dans data/referentiel, puis préparez les référentiels. Sans l'archive, npm run ban répond "archive absente" :

Windows	`Copy-Item $HOME\Downloads\adresses-33.csv.gz data\referentiel\` `npm run ban`
macOS et Linux	`cp ~/Downloads/adresses-33.csv.gz data/referentiel/` `npm run ban`

La force brute ne finit pas : --tailles a été oublié, le sujet tourne sur ses tailles par défaut. Relancez avec `--tailles 1,2,3`, comme au sous-objectif 3.

Les barèmes des critères sans chronomètre

- **Lisibilité** (poids 1) : la meilleure mesure reste de faire lire son code à quelqu'un qui ne l'a pas écrit. Ici, c'est votre cas : la force brute (legacy/src/adresses.ts) et les deux index (bench/variantes/adresses.ts) ont été écrits avant vous. Pour chaque option, répondez à trois questions : que fait la fonction, que se passe-t-il sur une saisie sans numéro, où changer le nombre de candidats. Listez les idées qu'il faut connaître pour répondre ; on part de 5 et chaque idée (la règle des trigrammes, un index inversé, une extension SQL) coûte un point.
- **Robustesse** (poids 3) : on exécute pour noter, `npm test` d'abord, puis l'exactitude par sorte sur trois cents saisies (sous-objectif 4). Barème : 5 si toutes les saisies sont retrouvées ; 4 si au moins 95 % le sont et si les échecs s'expliquent (une saisie sans numéro est ambiguë par nature, un nom de rue fréquent sans commune aussi) ; 3 entre 80 et 95 % ; 2 si la méthode se trompe sur le ticket T-0488 lui-même ou n'a été mesurée que sur dix saisies ; 1 si `npm test` échoue.
- **Coût de mise en oeuvre** (poids 2) : les lignes à écrire, les dépendances à ajouter en production, les données à préparer et à entretenir. Barème : 5 si rien ne change ; 4 pour quelques lignes sans donnée à préparer ; 3 pour quelques lignes et une donnée à reconstruire à chaque version du référentiel (un index) ; 2 pour une structure nouvelle à tenir dans la base (une extension à activer, une table et un index à entretenir) ; 1 si une dépendance s'ajoute au projet en production. PostgreSQL est déjà sur la VM d'Efrei (pièce 03) ; PGlite ne sert qu'à mesurer.
- Dans tous les cas, justifiez la note en une phrase sous la grille : le barème dit la note, la phrase dit ce que vous avez observé. Écrire le barème avant de mesurer n'interdit pas d'exécuter le code pour noter : ce qui s'écrit avant, c'est la règle ; la mesure vient ensuite.

Où trouver un exemple dans le code

bench/variantes/adresses.ts contient la règle des trigrammes (trigrammes), la similarité, garderMeilleurs, la construction de l'index et la variante PostgreSQL, écrites en entier. legacy/src/adresses.ts contient la force brute, lignes 43 à 45.

Si ça coince

- La commande s'arrête sur Cannot find package '@electric-sql/pglite' : npm install n'a pas été lancé après maj-J4.zip (annexe).
- Le correcteur dit que 19 saisies exactes sur 20 ne retrouvent pas leur adresse : les compteurs ne sont pas remis à zéro après chaque saisie.
- Vos candidats diffèrent de ceux de PostgreSQL (npm run exactitude -- --saisie les affiche côte à côte) : la similarité n'est pas calculée avec similarite(), ou son premier argument n'est pas le compteur de l'adresse.

Toujours bloqué après ces pistes : levez la main, ou écrivez sur Teams.

Si vous avez fini

- Mesurez une saisie sur la Gironde entière, trois minutes environ, puis mettez ce temps dans la grille : `npm run exactitude -- --referentiel gironde --requetes 100 --requetes-brute 2`.
- Multipliez le temps d'une saisie par les 600 adresses d'une nuit ordinaire, puis par les 6 412 de la nuit du 24 septembre (pièces 04 et 06) : écrivez-le en rubrique 5.

Les sorties montrées sont de vraies sorties, capturées en jouant la fiche atelier en cours ; vos durées seront différentes, vos nombres de carrefours, d'adresses retrouvées, de tournées et vos distances non.